

LLMs versus the Halting Problem: Characterizing Program Termination Reasoning

Oren Sultan^{1,2*}, Jordi Armengol-Estapé¹, Pascal Kesseli¹, Julien Vanegue^{3,4}
Dafna Shahaf², Yossi Adi^{1†}, Peter O’Hearn^{1,5†}

¹FAIR Team, Meta AI

²The Hebrew University of Jerusalem, Israel

³Bloomberg, New York, USA

⁴Imperial College London, UK

⁵University College London, UK

Abstract

Determining whether a program terminates is a central problem in computer science. Turing’s Halting Problem established termination as undecidable, showing that no algorithm can universally determine termination for all programs and inputs. Hence, verification tools approximate termination, sometimes failing to prove or disprove; these tools rely on problem-specific architectures, and are usually tied to particular programming languages. Recent advances in LLMs raise a natural question: *To what extent can they reason about program termination?* We evaluate frontier LLMs on a diverse set of C programs from the International Competition on Software Verification (SV-COMP) 2025. Our results show that GPT-5 and Claude Sonnet-4.5 achieve scores comparable to top-ranked verification tools (with test-time scaling). However, while models often correctly infer whether programs terminate, they frequently fail to construct a *witness* as formal proof, revealing a gap between semantic recognition and symbolic proof generation. Performance further degrades as code length increases. To analyze this gap, we introduce a divergence-precondition formulation that characterizes non-termination conditions as logical constraints. We hope these findings motivate future research on real-world termination benchmarks, neuro-symbolic approaches that combine LLMs with symbolic verification methods, and, more broadly LLM reasoning on other undecidable problems.

1 Introduction

Program termination is a central problem in computer science, demonstrated by the Halting Problem (Turing et al., 1936). The undecidability of the Halting Problem proves that no algorithm can universally determine whether a given program will halt. Ensuring program termination is important

for software reliability and safety (Avizienis et al., 2004). Non-termination may present a critical software defect that can lead to unbounded resource consumption, loss of system responsiveness, and severe failures in real-world deployments.

Despite the undecidability of the Halting Problem, a variety of verification tools have been proposed over the years to approximate termination analysis. State-of-the-art tools, such as UAutomizer (Heizmann et al., 2023) and PROTON (Metta et al., 2024), use automata over program statements or leverage symbolic execution via CBMC (Kroening and Tautschnig, 2014). These tools often rely on complex, multi-stage problem- and language-specific architectures (for example, see PROTON architecture in Figure 1).

Large Language Models (LLMs) have recently shown strong capabilities in code generation and reasoning (Zheng et al., 2023; Zhao et al., 2023; Liu et al., 2024; Jiang et al., 2024). Unlike traditional tools, LLMs can be integrated directly into the coding process, reason beyond surface-level syntax, and generalize across languages. We ask: *To what extent can frontier LLMs correctly predict whether a program terminates?* and evaluate standalone frontier LLMs on full SV-COMP termination tasks under machine-validated proof constraints.

Program termination is undecidable: proving it requires ruling out all infinite execution paths, not merely observing finite traces. Assessing termination requires LLMs to reason about program structure and control flow beyond pattern matching.

In this work, we assess the reasoning capabilities of LLMs on the International Competition on Software Verification (SV-COMP) 2025 Termination category¹ (Beyer, 2012). We cover both open-weights models, i.e., CWM (Copet et al., 2025) and Qwen3-32B (Yang et al., 2025) and proprietary models, i.e., GPT-5 (OpenAI, 2025), Claude Son-

*Work done during an internship at Meta FAIR.

†Joint supervision.

¹The dataset is publicly available under the CC-BY-4.0.

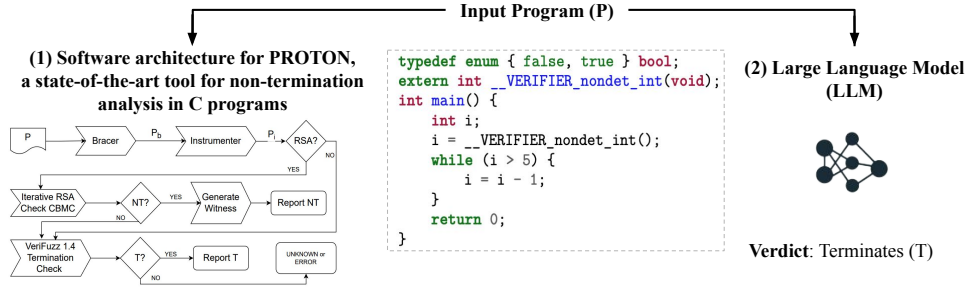


Figure 1: **Termination prediction task.** Given a program, the task is to predict whether the program halts for all inputs (instantiations of nondeterministic variables). (1) Traditional verification tools like PROTON use multi-component architectures for parsing, input augmentation, and tool-chain management (its architecture image reproduced from Metta et al. (2024) under Attribution 4.0 International License). (2) We investigate whether LLMs can match state-of-the-art verification tools, providing a simpler, language-agnostic solution, as illustrated in the example above where the LLM correctly predicts termination. For an example of non-termination see Figure 2.

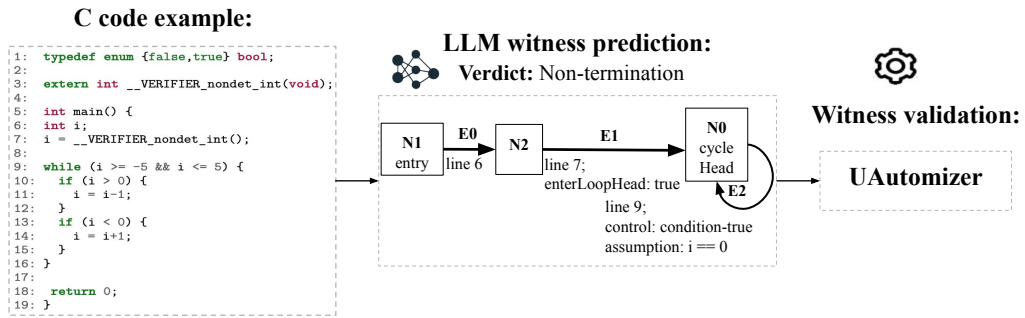


Figure 2: **LLM non-termination witness prediction.** Given a C program, an LLM that predicts non-termination must additionally output a **witness automaton** as a proof in JSON format (see Figure 8). The witness automaton models a potentially infinite execution: nodes correspond to program states and edges to transitions. The predicted JSON is converted to GraphML and validated by a **witness validator** (e.g., UAutomizer). The example illustrates a loop where i is initialized in $[-5, 5]$, eventually reaches 0 (see the *assumption* on edge E2), and executes indefinitely.

net 4.5 (Anthropic, 2025), and GPT-4o (Hurst et al., 2024), which serves as a non-reasoning baseline model. For a reference, we report results for the top three SV-COMP 2025 verification tools: PROTON (Metta et al., 2024), UAutomizer (Heizmann et al., 2023), and AProVE (Emrich et al., 2023).

Results indicate that GPT-5 and Claude Sonnet-4.5 reach performance levels trailing only the PROTON tool, which placed first. CWM ranks immediately behind UAutomizer, which finished second in SV-COMP. Error analysis reveals a substantial classification-proof gap: while LLMs are often highly effective at predicting program termination, they frequently fail to construct verifier-valid witness automata for infinite execution paths. Additionally, their performance tends to degrade when handling longer and more complex code samples.

To further validate our findings, we analyze a subset of programs by prompting the model to specify a *precondition* (a logical formula) describing the input values that cause non-termination. This is

related to the concept of weakest precondition from program verification (Dijkstra, 1976). We believe that such divergence preconditions are simpler and more interpretable than SV-COMP witnesses.

Note that our goal is to evaluate current LLMs on program termination prediction, rather than optimize them for this task. We believe that understanding these strengths and limitations can shed light on the broader capabilities of LLMs for challenging computational reasoning tasks. We view program termination as a useful testbed for studying frontier LLM reasoning on undecidable semantic tasks, motivating future research on real-world termination benchmarks, neuro-symbolic systems that combine LLMs with symbolic verifiers, and reasoning about additional undecidable properties.

Termination is one instance of a large class of undecidable semantic program properties. By Rice’s Theorem (Rice, 1953), every non-trivial semantic property of programs is undecidable, including central verification tasks such as unreachable,

memory safety, and program equivalence.

Our contributions are: (1) We present the first large-scale study of frontier LLM reasoning on full SV-COMP termination tasks, providing a new testbed for studying semantic reasoning on undecidable problems. (2) We show that frontier reasoning models approach specialized symbolic analyzers in prediction accuracy, while revealing a substantial gap between correct semantic prediction and valid symbolic proof construction. (3) We identify key failure modes of LLM reasoning: witness generation remains the primary bottleneck, performance degrades with code length, and errors concentrate in concurrency over standard control flow. (4) We introduce a domain witness formulation for non-termination, enabling a simpler and more interpretable analysis of model reasoning.

2 Problem Formulation

We follow the format specification of the SV-COMP Termination category, in which the **input** is a *C* program with one main function and possibly auxiliary helper functions. The program may include multiple non-deterministic variables, which can be assigned random values during execution. The **output** is a termination prediction. Specifically, we consider three possible predictions: (1) **T (Terminating)**, meaning the program terminates on all possible input values; (2) **NT (Non-Terminating)**, indicating the program diverges on some values and forms a non-termination loop. Here, the model is required to provide a *witness automaton*, which is a graph containing a cycle of valid states to demonstrate an infinite execution path within the program. Nodes correspond to program states; edges represent transitions. Importantly, this path does not need to be fully explicit; the model is not required to specify the exact non-deterministic values chosen along the path. As a result, the model does not need to produce a concrete counterexample, but rather a valid infinite path through the program’s state space. This allows standard SV-COMP validators to confirm correctness in under a second. See Appendix E for details about UAutomizer, the witness validator we used, and Figure 2 for an example of LLM witness prediction. (3) **UNK (Unknown)** is returned when the model cannot determine the termination behavior.

3 Dataset

We use SV-COMP 2025, which is arguably the largest public collection of formal verification tasks, spanning six key properties for tool evaluation. We focus on the termination category resulting in a total of 2,328 tasks. These tasks are organized into four subcategories: BitVectors (bit-precise arithmetic), MainControlFlow (complex control flow such as loops and recursion), MainHeap (dynamic heap memory manipulation), and Other. Each subcategory presents distinct program analysis challenges. A detailed summary of these subcategories is provided in Table 7 in the Appendix. Each task in the dataset provides a *C* code and its expected verification result, i.e., whether the code terminates or not. The benchmarks range from synthetic cases targeting specific algorithms to real-world code. Figure 2 illustrates a non-termination sample from the SV-COMP dataset.

Dataset statistics. The dataset contains *C* programs of diverse lengths. Using the Instruct Llama3 Tokenizer we find that 90% of samples include maximum of 20K tokens (median is 6K). See Appendix B for details. The dataset contains 2,328 samples, 65.6% labeled as Termination (T) and 34.4% as Non-termination (NT). This distribution is skewed toward terminating programs, reflecting typical real-world software; non-terminating cases often arise from unintended behaviors or bugs, which are also deliberately included in SV-COMP.

4 Experimental Setup

We provide a detailed description of the evaluation setup, covering model inference, evaluation methodology, and task-specific evaluation metrics.

4.1 Model Inference

We evaluate CWM (Copet et al., 2025), Qwen3-32B (Yang et al., 2025), GPT-5 (OpenAI, 2025) and Claude Sonnet-4.5 (Anthropic, 2025), all in reasoning mode. We additionally evaluate GPT-4o (Hurst et al., 2024) as a strong baseline model with no reasoning capabilities.

Inference pipeline. Each model is prompted to generate a JSON object containing a verdict attribute: true for Termination (T), and false for Non-termination (NT). For NT cases, the model also outputs a witness attribute, which is a JSON graph (nodes are states in the program; edges the transitions between states) representing a potential infinite path. We use the same prompt across

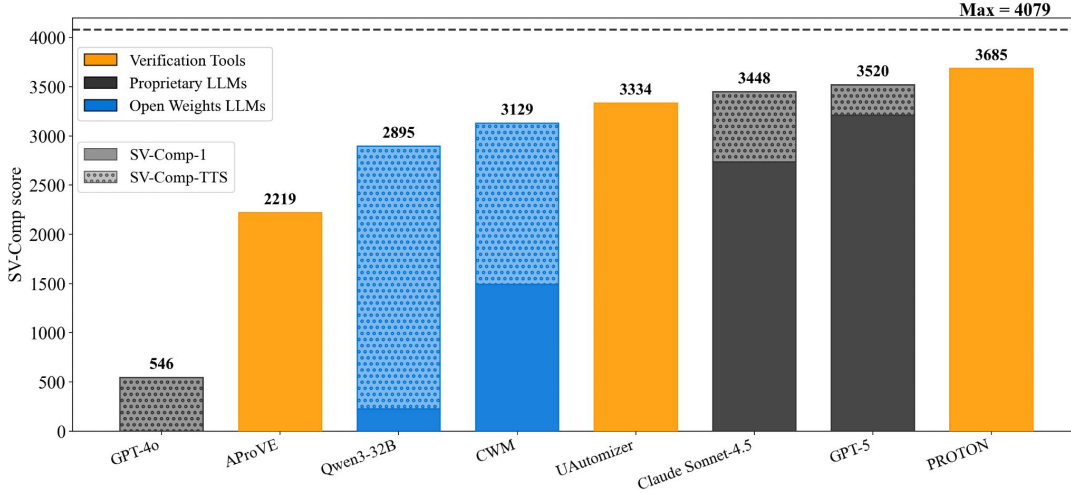


Figure 3: **SV-COMP main results.** Mean SV-COMP scores for LLMs (across 100 bootstraps), top SV-COMP 2025 verification tools, and the max possible scores (**minimum score is** $-50,064$). GPT-5 (TTS) and Claude Sonnet-4.5 (TTS) would place 2nd and 3rd (3,520 and 3,448), behind the gold medalist PROTON. CWM ranks just below UAutomizer. GPT-4o scores significantly lower: 546 with TTS, and a negative score without TTS ($-5,145$).

all models to ensure a controlled comparison. We deliberately avoid task-specific tuning (e.g., fine-tuning or prompt engineering) to reflect the models’ general performance. Despite the lack of task-specific tuning, models still achieve strong performance. See Figure D in the Appendix for the complete prompt provided to the model.

For a false *verdict* (NT), the LLM outputs a witness automaton graph in the *witness* JSON attribute. Predictions are schema-validated: nodes and edges must have unique ids and include required attributes (id, source, target, line², sourcecode). Duplicates or missing fields treated as errors. These ensure that outputs are syntactically valid and schema-compliant before validation; most failures arise during downstream semantic validation rather than at this stage (see Section 5.2). See Figure 8 in Appendix for an example. Finally, we convert the validated JSON object into a GraphML, which is then processed by a witness validator. We use the UAutomizer (Heizmann et al., 2023), which symbolically analyzes the candidate infinite path by accumulating constraints and using an SMT solver to validate the feasibility of an infinite execution. See Appendix E for details.

4.2 SV-COMP Score

The SV-COMP score averages performance across k categories with equal weight: $Score = \frac{1}{k} \sum_{i=1}^k \frac{s_i}{n_i} \left(\sum_{i=1}^k n_i \right)$, where s_i and n_i are the to-

²We add code line numbers supporting line prediction.

Prediction type	Score
Correct Termination (TN)	+2
Correct Non-termination (TP_valid_witness)	+1
Unknown	0
Correct Non-termination (TP_invalid_witness)	0
Incorrect Non-termination (FP)	-16
Incorrect Termination (FN)	-32

Table 1: **SV-COMP scoring per sample.** Non-termination (NT) is the positive class. For NT predictions, scores differ based on witness validity.

tal score and number of samples in category i .

Scoring per sample. We use the asymmetric scoring, following the metric used in SV-COMP, with the non-termination (NT) class treated as positive. See the scoring criteria in Table 1. This scheme prioritizes safety in verification. FN (misclassify diverging programs as terminating) can lead to deploying software with infinite loops, risking system failures and resource exhaustion.

4.3 Evaluation

Bootstrap sampling. LLM outputs are inherently stochastic, so repeated runs may yield different predictions. To ensure a robust evaluation, we employ bootstrap sampling. For each instance, we aggregate results across 100 bootstrap runs.

SV-COMP-1. We generate 20 predictions per instance and randomly select one prediction from the 20 generated for each bootstrap iteration. **SV-COMP-TTS.** To improve reliability and uncertainty estimates, we apply Test-Time Scaling (TTS) with

Consensus Voting. For each instance, we randomly select $n = 10$ out of 20 predictions. If all sampled votes agree (considering T and NT, while ignoring unknown), we adopt the unanimous prediction. If there is any disagreement, we output unknown to reflect model uncertainty, reducing the risk of penalties from incorrect predictions.

We found $n = 10$ as a sweet spot for TTS, representing an optimal trade-off. Lower values of n make consensus easier and reduce unknown outputs, while higher values increase the likelihood of disagreement and thus more unknown predictions, which can lower the overall score.

Note that requiring agreement between all trajectories is a strict criterion that may lead to excessive unknown predictions and reduced overall score. Future work could explore alternative agreement thresholds to optimize performance.

F1 score. Beyond the SV-COMP score, we report per-class F1 scores to enable a more fine-grained analysis of model performance. Unknowns predictions considered as a mistake.

Model hyper-parameters. We configured each model using its official recommended settings. Top-p was set to 0.95. Temperatures are set to 1.0 (CWM, Claude), 0.6 (Qwen), and 0.7 (GPT-4o). GPT-5 does not support temperature; we use the default configuration with medium reasoning effort.

Compute considerations. Proprietary models are accessed via API, while open-weight models run on our internal GPUs under a fixed budget. We rely on official SV-COMP 2025 results (Beyer and Strejček, 2025), evaluated under fixed limits (15 min CPU, 4 cores, 15 GB RAM). Note that GPU/API inference is generally more expensive than the CPU-based alternatives; we focus on *absolute* performance rather than cost-normalized comparisons.

5 Results

5.1 Main Results

Results are presented in Figure 3. Under the TTS setup, all reasoning models show impressive SV-COMP scores. GPT-5 (3,520) and Claude Sonnet-4.5 (3,448) achieve scores in the range of the top SV-COMP tools, trailing only PROTON. CWM (3,129) and Qwen-32B (2,895) follow, and do not exceed UAutomizer. GPT-4o, serving as a non-reasoning baseline, shows substantially lower performance. While all models benefit from TTS, this benefit is more pronounced in CWM, Qwen-32B, and GPT-4o (e.g., Qwen SV-COMP-1 is 225 while

Model	F1 (T) ↑	F1 (NT) ↑
GPT-5 (TTS)	0.98	0.96
GPT-5	0.98	0.97
Claude S-4.5 (TTS)	0.94	0.89
Claude S-4.5	0.98	0.96
CWM (TTS)	0.84	0.63
CWM	0.91	0.80
Qwen3-32B (TTS)	0.83	0.59
Qwen3-32B	0.90	0.78
GPT-4o (TTS)	0.73	0.50
GPT-4o	0.73	0.49

Table 2: **LLMs F1 scores per class.** F1 (T) and F1 (NT) are averaged over 100 bootstrap runs. F1 decreases under TTS due to increased unknown predictions (counted as errors in F1). Rankings align with SV-COMP score.

Model	P ↑	R ↑	V ↑
GPT-5	40.1%	40.7%	41.4%
Claude Sonnet-4.5	38.5%	39.9%	40.7%
CWM	28.5%	25.9%	30.6%
Qwen3-32B	22.6%	17.9%	24.8%
GPT-4o	20.8%	22.7%	27.3%

Table 3: **LLMs witness prediction (validated by UAutomizer) success rates.** Metrics: **P** (Precision): (%) of validated witness NT predictions out of all NT predictions; **R** (Recall): (%) of validated witness NT predictions out of all NT samples; **V** (Validity): (%) of validated witness NT predictions out of all correct NT predictions. GPT-5 and Claude lead in prediction rates, while CWM, GPT-4o, and Qwen3-32B perform lower, highlighting the challenge of generating a valid witness.

its SV-COMP-TTS is 2,895). Interestingly, the gap is the smallest for GPT-5, implying better model calibration and robustness across predictions.

Beyond the main SV-COMP score, Table 2 reports per-class F1 scores for both T and NT. The F1 ranking closely matches the SV-COMP ranking, with GPT-5 performing best overall. F1 scores decrease under TTS because additional unknown predictions are neutral in SV-COMP scoring but treated as errors in F1. Standard deviations remain consistently low (< 0.1), indicating stable performance across runs. See Appendix J for details.

5.2 Analysis

Next, we evaluate models along four axes: **(1)** witness validity; **(2)** Unknown rate (SV-COMP-1, TTS); **(3)** performance vs. code length; and **(4)** performance across subcategories.

Witness automaton prediction. Table 3 summarizes the results of witness automaton prediction, as

Model	Unk ↓	TTS-Unk ↓
GPT-5	0.5%	3%
Claude Sonnet-4.5	0.3%	8%
CWM	6%	22%
Qwen3-32B	2%	23%
GPT-4o	26%	30%

Table 4: **LLMs unknown prediction distribution.** **Unk:** (%) of unknown predictions, aggregated over 20 generations per entry. **TTS-Unk:** (%) of cases without unanimous agreement among 10 model predictions, resulting in an unknown. As we can see, GPT-5 and Claude Sonnet-4.5 rarely predict unknown, while GPT-4o does so most often. For TTS-Unk, GPT-5 and Claude show high prediction unanimity, whereas CWM, Qwen3-32B, and GPT-4o have more disagreement – improving SV-COMP scores but lowers F1 scores.

validated by the UAutomizer witness validator. Performance is evaluated using three metrics, which correspond to the prediction types defined in Table 1. The first metric, Validity (V), measures the proportion of correct NT predictions that have a valid witness, relative to all *correct* NT predictions: $\text{Validity} = \frac{TP_{\text{valid}}}{TP_{\text{valid}} + TP_{\text{invalid}}}$. Similarly, Precision (P) and Recall (R) are defined as the proportion of correct NT predictions with a valid witness (out of all NT predictions in precision, and out of all NT samples in recall). Note that Precision and Recall also account for NT label prediction errors: false positives (FP) in Precision and false negatives (FN) in Recall, which are independent of whether the witness is valid. GPT-5 achieves the highest witness automaton validated predictions, with Claude performing similarly. CWM, GPT-4o, and Qwen3-32B exhibit lower performance, with CWM providing the best performance among the three.

To analyze failure modes, we distinguish between *formatting* errors (e.g., invalid schema or graph structure) and *content* errors. Formatting failures are relatively rare: GPT-5 and GPT-4o produce none, Qwen and Claude around 5%, and CWM around 25%. Most outputs successfully pass syntax validation, and failures are primarily content-related: witnesses are well-formed but rejected by the validator as semantically invalid.

This reveals a gap between semantic prediction and proof construction: models often predict non-termination correctly yet fail to produce valid witnesses. Similar gaps have been observed in theorem proving and mathematical reasoning, where models generate correct answers more reliably than verifier-valid derivations (Sultan et al., 2025). Mo-

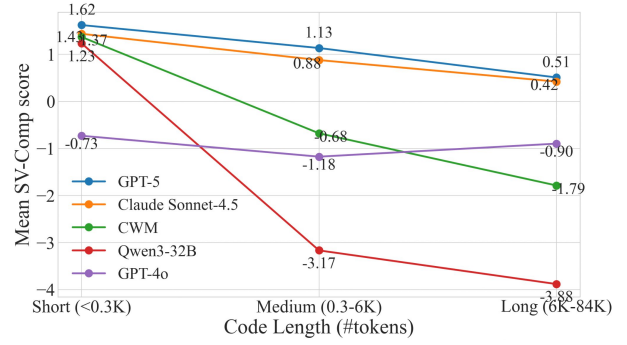


Figure 4: **LLMs mean SV-COMP score vs. code length.** Dataset examples are grouped into three *equal size* bins by code length (measured in tokens using Instruct Llama3 Tokenizer) (x axis). The mean SV-COMP score (y axis) per bin is shown for all models (20 predictions per sample). Scores decrease with code length; GPT-5 leads, followed by Claude and CWM.

tivated by this gap, we introduce an alternative witness format in Section 5.3.

Unknown prediction. Table 4 presents the results for unknown prediction considering both SV-COMP-1 and TTS. GPT-5 and Claude predict unknown least often, 0.5%, 0.3% respectively, followed by Qwen3-32B (2%) and CWM (6%). CWM’s higher unknown rate improves its SV-COMP-1 score compared to Qwen. GPT-4o predicts unknown much more frequently (26%), reflecting its difficulty with the classification task.

Under the TTS setup, GPT-5 and Claude have the lowest *unknown* rates (3% and 8%). These results are due to better consensus among model predictions, which implies better and more consistent model predictions. CWM, Qwen3-32B, and GPT-4o show less agreement, resulting in higher *unknown* rates, 22%, 23%, 30%, respectively.

Code input length. We analyze performance as a function of code complexity, using input length as a proxy (longer programs are typically more challenging). Figure 4 shows the mean SV-COMP score across three equal-size bins by input length (measured in tokens using the Instruct Llama3 tokenizer). Performance consistently decreases with length. GPT-5 achieves the highest scores across all bins, followed by Claude and CWM. In the medium and long bins, GPT-4o outperforms Qwen3 and matches CWM’s performance, although its scores are notably lower in the smallest bin. GPT-4o predicts a relatively high number of unknown outcomes, which helps reduce penalties for longer code inputs. This suggests that the main challenge is not only local code understanding, but main-

Model	Domain W. $\uparrow$		SV-COMP W. $\uparrow$	
	P@1	P@3	P@1	P@3
Claude S-4.5	0.87	0.96	0.80	0.83
CWM	0.82	0.96	0.75	0.83
Qwen3-32B	0.85	0.94	0.77	0.83

Table 5: **LLMs performance per witness formats.** We report Mean Pass@1 and Pass@3 for correct witness predictions on 40 SV-COMP NT samples (10 generations each) on these short code samples. All models perform well, with higher accuracy for Domain W.

taining globally consistent reasoning over longer control-flow structures and larger state spaces. Termination reasoning thus exposes a long-horizon semantic reasoning failure analogous to degradation observed in multi-step mathematical reasoning.

Sub-category error rates. We analyze GPT-5, the top-performing model. Errors are concentrated in multi-threaded programs, while simpler control-flow categories (e.g., loops, arrays, recursion) show near-zero failure rates. See Appendix G for details.

Evidence against data memorization. Several findings suggest that the results are unlikely to be explained by benchmark memorization. (1) Models recover non-termination domains in a novel witness format not available online (Section 5.3); (2) performance degrades with code length and complexity; (3) non-reasoning models perform substantially worse than reasoning-oriented models; and (4) anonymizing function and variable names does not affect performance (Appendix H).

5.3 Precondition Prediction

Since the SV-COMP witness format is rather complex, we propose an interpretable alternative in which models are asked to directly output a logical expression over non-deterministic variables characterizing conditions for non-termination (e.g., $x < 0 \wedge y = 0$ indicates that the program diverges only when x is negative and y is zero). This can be seen as the negation of the weakest precondition of (Dijkstra, 1976) (taking “true” as the post), and can be viewed as the weakest precondition of the divergent Hoare triple (Raad et al., 2024).

To assess models’ ability to predict logical expressions representing non-termination (NT), we manually annotated 40 NT programs from the SV-COMP dataset. Each program is labeled with a logical expression for the NT domain. One annotator labeled all samples, while four others annotated a disjoint subset. Annotators were instructed to

“Read the C program and specify a logical expression over the non-deterministic variables characterizing inputs leading to non-termination.”. All annotators exhibited perfect agreement with the first annotator. We restrict to short programs with at most two non-deterministic variables to keep annotation manageable and reliable. As a result, the dataset contains simple logical expressions. We acknowledge that more complex expressions could arise in general (e.g., equations with higher-order terms or non-linear constraints).

Since model predictions for NT domains may use different (syntactically) but equivalent logical expressions (e.g., $x < 10$ vs. $x \leq 9$), we use the Z3 theorem prover (De Moura and Bjørner, 2008) to verify equivalence with the ground truth. Table 5 reports mean Pass@1, and Pass@3 (Chen, 2021) over 40 annotated samples (10 predictions each). “Domain” reports our proposed format, while “SV-COMP” uses the graph-based format (Appendix I). Both achieve high accuracy, but Domain consistently outperforms SV-COMP in Pass@1 and Pass@3, indicating stronger support for valid witness generation. It is also more interpretable and token-efficient.

Examples of model witness outputs. We find that certain NT samples yield a valid domain witness format but not a valid graph-based format, and vice versa. This suggests that the two formats are complementary: errors in one format do not necessarily occur in the other. See Appendix K for examples of both formats, including one with the model’s reasoning process prior to its final answer.

6 Related Work

LLMs vs. classic reasoners. Kambhampati (2024) examine the capacity of LLMs for reasoning and planning, showing that they underperform classical planners on benchmarks from the International Planning Competition (Valmeekam et al., 2024). They argue that while LLMs may complement traditional planners through their language capabilities, they fall short in both accuracy and efficiency (Kambhampati et al., 2024). Similarly, Hazra et al. (2025) report that reasoning-oriented models yield improvements on SAT tasks but still do not reach the performance of classical solvers. In the context of theorem proving, prior work mostly integrates LLMs with symbolic reasoning systems such as Lean, SMT solvers, and first-order logic provers-where LLMs generate candi-

date proofs that are verified by symbolic tools, improving performance and reliability (Sultan et al., 2025; Trinh et al., 2024; Mirzadeh et al., 2024; Régis et al., 2024; Szeider, 2024; Olausson et al., 2023). In contrast, we evaluate the standalone ability of LLMs to reason about program termination, noting that practical systems should pair them with symbolic proof checkers.

Termination analysis using LLMs. Most prior work has focused on predicting program inputs and outputs (Gu et al., 2024; Xu et al., 2025). Recent studies show that jointly training models to predict code and execution traces improves code generation (Armengol-Estapé et al., 2025), and that learning code world models further extends this capability (Copet et al., 2025). Termination prediction goes beyond execution, requiring reasoning about the existence of non-terminating behaviors. Another line of work uses LLMs to synthesize termination arguments, such as ranking functions (Kamath et al., 2024), but, to our knowledge, does not address non-termination witnesses. Alon and David (2022) applies GNNs to termination reasoning, using timeouts as a heuristic proxy for non-termination. More recently, Copet et al. (2025) evaluate CWM and Qwen3 on a simplified subset of translated Python functions from SV-COMP and TPDB,³ showing strong performance.

Termination analysis tools. In SV-COMP 2025, the leading systems are PROTON (Metta et al., 2024) and UAutomizer (Heizmann et al., 2023). PROTON uses CBMC (Kroening and Tautschnig, 2014) to soundly detect NT via recurrent loop states and generate witnesses, and falls back on high-confidence termination checks from VeriFuzz (Metta et al., 2023). UAutomizer encodes programs as automata and applies SMT-based refinement to eliminate infeasible paths. SV-COMP tools typically target small, self-contained programs. In contrast, Raad et al. (2024) study repository-level termination over millions of lines of code using recurrent sets (Gupta et al., 2008), bi-abduction (Calcagno et al., 2011), and underapproximation (O’Hearn, 2020).

7 Discussion

Decidable vs. Undecidable problems. As discussed in Section 6, LLMs underperform classical solvers on decidable tasks such as SAT and propositional planning. Interestingly, our results suggest

that frontier LLMs can achieve competitive empirical performance with specialized tools on practical termination benchmarks, a task that is theoretically more challenging due to its undecidability. This discrepancy may be partly attributable to differences in benchmark construction: SAT and planning competition benchmarks are largely synthetically generated, whereas SV-COMP has evolved through community contributions. As a result, SV-COMP more closely reflects naturally occurring verification problems rather than being explicitly designed to be adversarial. Additionally, undecidable problems depend on heuristic reasoning rather than exhaustive search, making them potentially a more natural fit for LLMs.

Beyond SV-COMP. SV-COMP remains the most comprehensive and challenging benchmark for termination analysis. Prior work focused on less-challenging benchmarks, e.g., on Python programs translated from TPDB (Copet et al., 2025). However, we believe that making LLMs useful for program termination requires extending to even more challenging settings, and that the ultimate assessment of the practical utility of LLMs in termination analysis should derive from deployment on real-world codebases (e.g., within continuous integration). In practical deployments, automatic termination reasoning must handle a continual stream of previously unseen problems and meet the accuracy requirements of real-world engineering.

8 Conclusions and Future Work

We studied whether frontier LLMs can reason about program termination competitively with state-of-the-art verification tools on the SV-COMP benchmark. We empirically show strong performance of LLMs – GPT-5 and Claude Sonnet-4.5 (with TTS) achieve performance close to the top-ranked verifier PROTON, while CWM performs close to UAutomizer. Across all models, performance degrades as code length increases, and witness generation remains a notable challenge. To address this gap, we introduced a divergence-precondition formulation that characterizes non-termination conditions as logical constraints.

More broadly, our work highlights program termination as a useful testbed for understanding the strengths and limitations of frontier LLM reasoning, particularly the emerging gap between semantic recognition and formally verifiable reasoning.

In the future, we plan to construct challenging

³<https://github.com/TermCOMP/TPDB>

real-world termination benchmarks, explore neuro-symbolic approaches that combine LLMs with symbolic verifiers, and extend this line of research to additional undecidable reasoning problems.

Limitations

- **Programming Language.** Our evaluation focuses on SV-COMP, which consists exclusively of *C* programs. Notably, prior work suggests that LLM-based termination reasoning can transfer across languages, e.g., to Python programs translated from TPDB (Copet et al., 2025).
- **Proprietary LLM (OpenAI GPT-5).** GPT-5 achieves the best performance across models. While this model delivers strong performance, its architecture, training data, and training methodology are proprietary.
- **LLMs are sensitive to prompt phrasing.** LLMs are sometimes sensitive to small changes to the prompts. We acknowledge that results may vary with different prompts. We note that we use a simple standardized prompt without benchmark-specific tuning to avoid overfitting. Despite this minimal setup, models achieve competitive performance.
- **Lab versus competition conditions.** The LLM results reported on in this work were not from official entry to the SV-COMP competition, and exact competition conditions cannot be guaranteed, and this could affect comparison to the scores of SV-COMP entrants. We expect that our LLM results are conservative, as we use one witness validator rather than many. This work was completed after the deadline for SV-COMP 2026; perhaps LLM entries could participate in SV-COMP 2027.
- **Robustness evaluation.** We leave a more systematic robustness evaluation under semantics-preserving transformations, adversarial rewrites, procedurally generated benchmarks. Consequently, our findings should primarily be interpreted as a behavioral characterization of current frontier LLMs on SV-COMP-style termination tasks rather than definitive evidence of general semantic reasoning capabilities.

Ethical Considerations

AI Assistants. We used AI assistants for coding support, writing, and rephrasing. All AI-generated outputs were carefully reviewed and edited to ensure alignment with our design goals and to preserve the original intent of our work.

Acknowledgments

We thank Ori Yoran for his valuable feedback, as well as the whole CodeGen team at FAIR for helpful discussions. We thank Marco Selvi and Kunhao Zheng for the initial integration of the uAutomizer witness checker.

References

- Yoav Alon and Cristina David. 2022. [Using graph neural networks for program termination](#). In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, ESEC/FSE 2022, Singapore, Singapore, November 14-18, 2022*, pages 910–921. ACM.
- Anthropic. 2025. [Claude sonnet 4.5](#).
- Jordi Armengol-Estapé, Quentin Carbonneaux, Tianjun Zhang, Aram H Markosyan, Volker Seeker, Chris Cummins, Melanie Kambadur, Michael FP O’Boyle, Sida Wang, Gabriel Synnaeve, and 1 others. 2025. What i cannot execute, i do not understand: Training and evaluating llms on program execution traces. *arXiv preprint arXiv:2503.05703*.
- Algirdas Avizienis, J-C Laprie, Brian Randell, and Carl Landwehr. 2004. Basic concepts and taxonomy of dependable and secure computing. *IEEE transactions on dependable and secure computing*, 1(1):11–33.
- Dirk Beyer. 2012. Competition on software verification: (sv-comp). In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, pages 504–524. Springer.
- Dirk Beyer and Jan Strejček. 2025. Improvements in software verification and witness validation: Sv-comp 2025. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*, pages 151–186. Springer.
- Cristiano Calcagno, Dino Distefano, Peter W. O’Hearn, and Hongseok Yang. 2011. [Compositional shape analysis by means of bi-abduction](#). *J. ACM*, 58(6):26:1–26:66.
- Mark Chen. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.
- Jade Copet, Quentin Carbonneaux, Gal Cohen, Jonas Gehring, Jacob Kahn, Jannik Kossen, Felix Kreuk, Emily McMilin, Michel Meyer, Yuxiang Wei, and